

Datoviz v0.4: a C-first GPU rendering engine for scientific visualization

Cyrille Rossant ¹

¹ International Brain Laboratory

DOI: [N/A](#)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: 01 January 1970

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Datoviz is an open-source engine for interactive, GPU-accelerated visualization of large scientific datasets. It renders dense two- and three-dimensional data — point clouds, images, meshes, volumes, text, and annotations — at interactive frame rates, and it can be driven directly from C or from Python. A single scene can hold on the order of millions of primitives: the bundled examples include a colorized LiDAR point cloud of up to eight million points, a cortical-surface mesh of up to one million vertices, an interactive embedding of two hundred thousand queryable points, and a live electrophysiology stream of 64 continuously updating traces ([Figure 1](#)). The same scene can be shown in a desktop window, rendered off-screen to reproducible images or video, or run experimentally in a web browser through WebGPU.

Version 0.4 is a substantial rewrite that repositions Datoviz as a C-first rendering *engine* rather than a high-level Python plotting package. Earlier releases exposed an object-oriented plotting API; v0.4 instead places a small, stable C application binary interface (ABI) at its core, adds a retained “scene” model of figures, panels, and visual objects on top of it, and generates Python bindings (ctypes) that call the same ABI and accept NumPy arrays directly. This makes Datoviz suitable for embedding inside scientific applications and for use as a rendering backend beneath higher-level tools, rather than as a one-line plotting command for end users.

The v0.4 surface provides retained visual families for points, pixels, markers, segments, paths, vectors, primitives, images, text, labels, meshes, spheres, and volumes — with experimental glyph and splat families — together with panels, cameras and interactive controllers, axes, colorbars, scale bars, picking and readback queries, and off-screen capture. The goal is not to maximize the number of plot types but to provide a compact, reusable engine whose rendering behavior is shared across native applications, Python code, generated examples, and browser demonstrations.

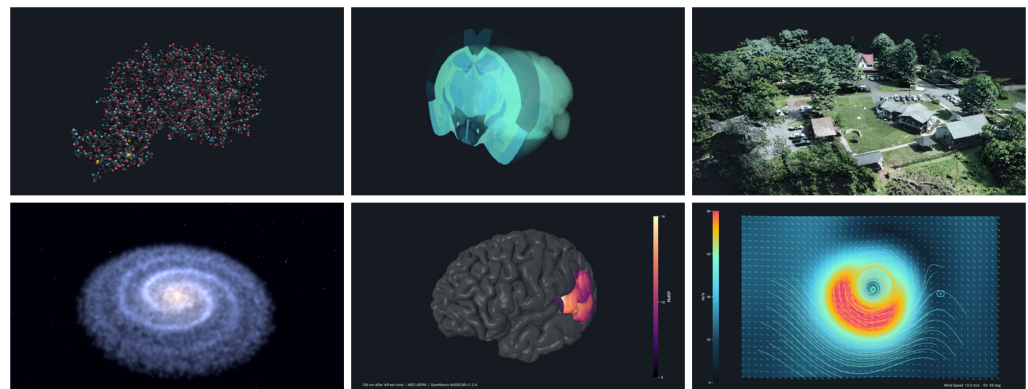


Figure 1: A selection of Datoviz showcase scenes, each rendered by the v0.4 engine and reproduced from the example gallery. Top row: a molecular structure drawn with sphere impostors, an Allen mouse-brain volume rendering, and a colorized LiDAR point cloud. Bottom row: an animated galaxy particle field, cortical activity on a one-million-vertex surface mesh, and a geophysical wind vector field.

Statement of need

Many scientific datasets are too large, dynamic, or interactive for static figure-generation workflows. Neuroscience, microscopy, physics simulation, geospatial analysis, and computational modeling routinely require dense point clouds, meshes, image stacks, labels, axes, annotations, linked views, and camera control while preserving a path to reproducible image and video capture. General-purpose plotting libraries serve many publication figures well, but the authors of domain-specific viewers and reusable plotting backends need a lower layer that exposes predictable rendering semantics, efficient data updates, embedding hooks, and explicit control over GPU resources.

Datoviz addresses this engine-layer need. It targets scientific software developers who build interactive applications, domain-specific viewers, or reusable plotting backends, rather than end users who want a single plotting call. This role motivated the v0.4 rewrite. The earlier releases, described by Rossant et al. (2021), coupled rendering to a Python plotting API, which made the engine hard to embed and to reuse from other languages or higher-level libraries. Version 0.4 separates the two concerns: Datoviz now owns the C engine, the retained scene and application path, the generated Python binding surface, raster and video capture, an experimental browser path, and a narrow compute-to-render capability, while high-level object-oriented plotting is expected to live in projects above Datoviz such as the Graphics Server Protocol (GSP) and future VisPy2-style layers. Because v0.4 does not preserve the v0.3 source, ABI, or Python plotting API, it is best treated as a new software release rather than an incremental update of the earlier work.

Datoviz is developed in the context of the International Brain Laboratory, where interactive inspection of large neuroscience datasets is a recurring software need, and its example gallery spans neuroscience, geospatial data, astronomy, molecular structure, and physical simulation. The project can be cited directly as the rendering engine of a scientific application, or as the backend layer beneath a higher-level or domain-specific tool.

State of the field

Scientific visualization tools span a wide range of abstraction levels. Matplotlib is the standard for publication-oriented static and interactive figures (Hunter, 2007), while Plotly and Bokeh target interactive, web-based charts (Bokeh Development Team, 2018; Plotly Technologies Inc., 2015). VisPy pioneered GPU-accelerated visualization in Python and directly influenced

Datoviz's original design (Campagnola et al., 2015); the more recent pygfx and fastplotlib build modern GPU rendering engines on WebGPU for the same community (fastplotlib contributors, n.d.; pygfx contributors, n.d.). napari provides a successful domain-oriented viewer for multidimensional image data (napari contributors, 2019), and VTK and ParaView offer broad visualization pipelines for meshes, volumes, and simulation data (Ahrens et al., 2005; Schroeder et al., 2006). At the graphics-API level, Vulkan provides low-overhead native GPU access (Khronos Group, n.d.) and WebGPU is standardizing portable GPU access for browsers and applications (World Wide Web Consortium, n.d.).

Datoviz does not try to replace these tools at their own level; its niche is architectural. It provides a small, embeddable, C-ABI-oriented engine with a single retained scene contract that can drive native rendering, low-level Python bindings, capture paths, and experimental browser execution. Folding this role into a high-level plotting package would entangle user-facing concerns with backend and runtime ownership, while adopting a large visualization framework as the sole substrate would make it harder to preserve the explicit C-first API that downstream integrations require. Datoviz therefore occupies a narrow middle layer — lower than plotting libraries, more specialized than a raw graphics API — and continues a lineage of GPU visualization work in this community that includes Glumpy and Galry alongside VisPy.

Software design

The central v0.4 design choice is to keep rendering semantics above the graphics backend but below high-level plotting. User code creates scenes, figures, panels, controllers, and visuals. Scene emission produces frame plans, which are lowered to command streams — an internal rendering protocol, DRP2 — that are consumed by the native Vulkan runtime and, for a declared subset, by the WebGPU/WASM runner. This separation makes scene behavior testable without forcing every check through a live window, and it gives the browser backend a shared contract instead of a separate JavaScript reimplementations of examples.

The C ABI is the stable center of the engine. Public headers define the supported scene, visual, application, runtime, and FFI-helper surfaces, while internal implementation remains in modular subsystems. The Python story follows that boundary: `datoviz.raw` is generated from the exported C ABI and preserves exact ctypes signatures, while the top-level facade (`import datoviz as dvz`) keeps C-shaped names but adapts NumPy array arguments for declared data-upload calls. This avoids making a v0.3-compatible plotting API a compatibility constraint on v0.4.

The design deliberately trades some convenience for explicitness. GPU ownership rules, borrowed handles, framebuffer sizing, and readback lifetimes are documented rather than hidden behind a large object model, and native presentation, off-screen capture, and browser execution share the same scene-to-command-stream boundary. WebGPU support is scoped as an experimental subset, and compute support is limited to a narrow compute-to-render path rather than a general compute framework, which keeps the release surface easier to validate. Examples are treated as part of the design: they are organized by visual family, feature, and showcase, and many browser routes reuse the same canonical C scenario as native validation, keeping documentation, tests, and release evidence aligned with actual engine behavior.

Availability and reproducibility

Datoviz v0.4 is released under the MIT license, with source code, documentation, and an example gallery at <https://datoviz.org>. It installs from PyPI as a binary wheel — the current release candidate is 0.4.0rc2 — and can also be built from source and consumed as a C/C++ library through CMake and pkg-config. The published v0.4.0rc2 wheels cover Linux (x86_64 and aarch64), macOS 15 (arm64 and Intel), and Windows (AMD64 and ARM64) across Python 3.10 through 3.14.

The release is designed for reproducible evaluation. The repository includes focused tests, generated C and Python binding checks, native examples, WebGPU fixtures and browser smoke tests, wheel build and installed-consumer checks, and gallery examples that double as validation artifacts. Explicit status pages classify each visual family, feature, backend, and optional provider as supported, experimental, advanced, deferred, or external, so that users and reviewers can tell the stable surface from work in progress. A version-specific Zenodo archive and DOI will accompany the final v0.4.0 release to provide a citable software record.

AI usage disclosure

Development and release of Datoviz v0.4 were assisted by [OpenAI Codex](#) and [Anthropic Claude Code](#), including implementation, refactoring, test development, debugging, documentation, release validation, citation metadata, release-checklist text, and drafts of this paper. The human author directed and reviewed this work and is responsible for the software design, implementation decisions, citations, claims, and submitted text. AI-assisted content was reviewed, edited, tested where applicable, and validated against the repository state and cited sources.

Acknowledgements

Datoviz development has been supported by the International Brain Laboratory, the Wellcome Trust, the Simons Foundation, and the Chan Zuckerberg Initiative.

References

- Ahrens, J., Geveci, B., & Law, C. (2005). ParaView: An end-user tool for large data visualization. *The Visualization Handbook*.
- Bokeh Development Team. (2018). *Bokeh: Python library for interactive visualization*. <https://bokeh.org/>
- Campagnola, L., Klein, A., Larson, E., Rossant, C., & Rougier, N. P. (2015). VisPy: Harnessing the GPU for fast, high-level visualization. *Proceedings of the 14th Python in Science Conference (SciPy 2015)*, 91–96. <https://doi.org/10.25080/Majora-7b98e3ed-00e>
- fastplotlib contributors. (n.d.). *Fastplotlib: A fast, interactive scientific plotting library*. <https://github.com/fastplotlib/fastplotlib>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Khronos Group. (n.d.). *Vulkan API specification*. <https://registry.khronos.org/vulkan/>
- napari contributors. (2019). *Napari: A multi-dimensional image viewer for python*. <https://doi.org/10.5281/zenodo.3555620>
- Plotly Technologies Inc. (2015). *Plotly: Collaborative data science platform*. <https://plotly.com/>
- pygfx contributors. (n.d.). *Pygfx: A WebGPU-based render engine*. <https://github.com/pygfx/pygfx>
- Rossant, C., Rougier, N. P., Comba, J., & Gaither, K. (2021). High-performance interactive scientific visualization with datoviz via the vulkan low-level GPU API. *Computing in Science & Engineering*, 23(4), 85–90. <https://doi.org/10.1109/MCSE.2021.3078345>
- Schroeder, W., Martin, K., & Lorensen, B. (2006). *The visualization toolkit: An object-oriented approach to 3D graphics* (4th ed.). Kitware.

World Wide Web Consortium. (n.d.). *WebGPU*. <https://www.w3.org/TR/webgpu/>